

근로계약서 온체인-오프체인 아키텍처 정리본

1. 문서의 목적

이 문서는 외국인 노동자·고용주 대상 서비스에서 근로계약서를 안전하게 등록·처리·분석하기 위한 전체 아키텍처와 기본 플로우를 간단히 정리한 기준 문서이다.

현재 단계에서는 기술적 세부 구현보다 전체 방향과 역할 분리를 명확히 하는 것을 목표로 한다. 이후 백엔드 엔티티 설계, API 설계, OCR 서비스 분리, AI 레이어 연계 설계는 이 문서를 기준으로 구체화한다.

2. 문제 인식

근로계약서는 단순 문서가 아니라 개인정보와 계약 조건이 포함된 민감한 법적 문서이다. 따라서 다음 조건을 동시에 만족해야 한다.

- 계약서 원문을 안전하게 저장할 수 있어야 한다.
- 문서 위변조 여부를 검증할 수 있어야 한다.
- OCR을 통해 계약 내용을 구조화할 수 있어야 한다.
- 추후 AI 레이어가 문서 내용을 기반으로 설명·분석할 수 있어야 한다.
- 온체인 구조를 사용하더라도 개인정보 및 원문 노출 문제를 피해야 한다.

이 때문에 계약서 원문 자체를 블록체인에 저장하는 방식이 아니라, **원문은 오프체인에 저장하고 해시값만 온체인에 기록하는 구조**를 기본 방향으로 잡는다.

3. 핵심 설계 원칙

3-1. 온체인과 오프체인의 역할 분리

- **오프체인**: 실제 계약서 원문 저장, OCR 수행, 문서 구조화, 권한 관리
- **온체인**: 문서 해시값, 문서 등록 시점, 상태 이력 등 최소 증빙 정보 기록

즉, 온체인은 실제 문서를 처리하는 레이어가 아니라 **문서 무결성과 이력을 증명하는 레이어**로 본다.

또한 본 시스템은 문서의 “증빙(무결성)”과 “내용 해석(분석)”을 분리하여 설계하며, 온체인은 증빙을 담당하고, 오프체인 및 AI 레이어는 분석을 담당한다.

3-2. 원문은 온체인에 올리지 않는다

근로계약서 원문에는 개인정보와 민감한 계약 조건이 포함되므로, 원문 자체를 블록체인에 저장하는 것은 적절하지 않다. 또한 계약서는 수정·재등록·버전 관리가 필요할 수 있으므로, 변경이 어려운 온체인과도 성격이 다르다.

따라서 블록체인에는 원문 대신 **해시값과 최소 메타데이터만 기록**한다.

3-3. OCR 결과는 그대로 AI에 넣지 않는다

OCR 결과는 줄바꿈 깨짐, 표 손상, 숫자/단위 혼합, 다국어 혼재 등으로 인해 그대로 활용하기 어렵다. 따라서 OCR 이후에는 반드시 정규화·구조화 레이어를 거쳐야 한다.

3-4. 문서 등록과 문서 분석은 분리된 흐름으로 본다

문서 업로드와 증빙 기록은 문서 등록 파이프라인이고, OCR 및 분석 처리는 후처리 파이프라인이다. 이 둘을 분리하면 OCR 실패가 발생해도 문서 등록 자체는 정상적으로 유지할 수 있다.

4. 전체 아키텍처 개요

4-1. 주요 구성 요소

1. 클라이언트 레이어
2. 고용주: 계약서 업로드 및 등록
3. 근로자: 계약서 확인 및 서명/동의
4. Spring 백엔드
5. 업로드 요청 수신
6. 파일 메타데이터 저장
7. 해시 생성
8. 온체인 기록 요청
9. OCR 서비스 호출
10. 구조화 결과 저장
11. AI 레이어 인계용 데이터 생성 및 전달
12. 문서 상태 관리
13. 오프체인 스토리지
14. PDF, 이미지 등 계약서 원문 저장
15. 실제 문서 파일 보관
16. 온체인 레이어
17. 문서 해시값 기록
18. 등록 시점 및 상태 이력 기록
19. 무결성 검증 기준 제공
20. OCR 서비스

21. PaddleOCR / PP-Structure 기반 처리
 22. 텍스트 추출, 레이아웃 분석, 표 구조 인식
 23. 정규화 및 구조화 레이어
 24. OCR 결과 정리
 25. 문서 유형 분류
 26. 주요 필드 추출
 27. AI 입력용 구조화 데이터 생성
 28. AI 레이어
 29. 구조화된 데이터를 기반으로 계약 내용 해석 및 응답 생성
-

5. 기본 처리 플로우

5-1. 문서 등록 플로우

1. 고용주가 근로계약서 파일을 업로드한다.
2. Spring 백엔드는 파일을 수신한다.
3. 파일 원문을 오프체인 스토리지에 저장한다.
4. 문서 메타데이터를 저장한다.
5. 저장된 원문 기준으로 해시값을 생성한다.
6. 해시값과 최소 메타데이터를 온체인에 기록한다.
7. 문서 상태를 등록 완료 상태로 갱신한다.

이 단계의 핵심은 **문서를 안전하게 보관하고, 해당 시점의 문서 무결성을 증빙할 수 있게 만드는 것**이다.

5-2. OCR 및 구조화 플로우

1. 등록 완료된 문서를 OCR 처리 대상으로 넘긴다.
2. OCR 서비스가 문서의 텍스트와 레이아웃 정보를 추출한다.
3. 정규화 레이어에서 OCR 결과를 정리한다.
4. 문서 유형을 분류한다.
5. 주요 필드를 추출한다.
6. 구조화된 결과를 저장한다.
7. 문서 상태를 OCR 완료 상태로 갱신한다.

이 단계의 핵심은 **문서를 시스템이 해석 가능한 구조로 바꾸는 것**이다.

5.3. AI 레이어 인계 플로우

1. 구조화된 계약서 데이터를 AI 레이어로 전달한다.
2. AI 레이어는 전달받은 데이터를 기반으로 계약 내용 설명, 조항 분석, 사용자 맞춤 응답을 생성한다.
3. 백엔드는 인계 요청 및 처리 상태를 관리한다.

이 단계의 핵심은 **AI가 바로 활용 가능한 구조화된 입력 데이터를 안정적으로 전달하는 것**이며, 실제 해석 및 추론 로직은 백엔드가 아닌 AI 레이어의 책임으로 본다.

6. 문서 상태 흐름 예시

- `UPLOADED`
 - `STORED`
 - `HASHED`
 - `ANCHORED_ON_CHAIN`
 - `OCR_PROCESSING`
 - `OCR_COMPLETED`
 - `ANALYZED`
-

7. 역할 분리 관점에서의 정리

Spring 백엔드의 역할

- 전체 흐름 오케스트레이션
- 파일 업로드 처리
- 메타데이터 저장
- 해시 생성
- 온체인 연동
- OCR 서비스 호출
- 결과 저장 및 상태 관리
- AI 레이어 인계용 데이터 생성 및 전달

OCR 서비스의 역할

- 문서 분석
- 텍스트 및 레이아웃 추출

온체인 레이어의 역할

- 해시값 저장
- 문서 존재 시점 증빙

AI 레이어의 역할

- 구조화된 데이터를 기반으로 분석 및 응답 생성
-

8. 현재 단계에서의 구현 방향 요약

- 계약서 원문은 오프체인에 저장
- 해시만 온체인 기록
- OCR은 별도 서비스
- Spring은 오케스트레이터
- 구조화 결과만 AI 레이어로 전달

- 등록/분석 파이프라인 분리
 - 백엔드는 AI 인계 직전까지 책임
-

9. 향후 구체화 대상

1. 엔티티 설계
 2. API 설계
 3. OCR 인터페이스
 4. 온체인 어댑터
 5. 서명 흐름
 6. CASE 권한 모델
-

10. 결론

이 시스템은 **오프체인 중심 처리 + 온체인 증빙 구조**를 기반으로 하며, 백엔드는 문서 무결성과 데이터 가공까지 책임지고, AI 레이어는 해석과 응답을 담당하는 구조로 설계된다.